

Entregable Final: Migración de Tablas, Vistas y Consultas de Verificación

Migración de la base de datos employees de MariaDB a PostgreSQL

Fernando Adriel Montero Casanova

Tecnología de Base de Datos I

Docente: Jared Lopez Leaños

29 de septiembre de 2026

Santa cruz-Bolivia

Datos generales

Contexto

Este entregable continúa el proyecto de migración de la base `employees` iniciado en la actividad anterior del Bloque 3, donde se documentó el diccionario de datos, se diseñó el modelo en pgModeler y se migró la **estructura** (DDL) hacia la base `pdb_employees` en PostgreSQL 18, sin datos. En esta fase se migran los **datos**, las **vistas**, y se ejecutan **consultas de verificación** para confirmar que la migración fue exitosa y consistente.

> *Por buenas prácticas, las contraseñas se omiten o se muestran enmascaradas en los comandos de este informe.*

Repositorio del proyecto: https://github.com/fernando-420/entregable-final

Punto 1: Migración de Tablas

Nota metodológica: intento inicial con pgloader

Antes de la exportación manual, se intentó migrar los datos con `pgloader` en modo `data only` apuntando directamente a la estructura ya existente en `pdb_employees`:

```
LOAD DATABASE
    FROM mysql://root:********@127.0.0.1:3306/employees
    INTO postgresql://fernando:********@127.0.0.1:5432/pdb_employees

WITH data only, disable triggers,
    workers = 8, concurrency = 2,
    batch rows = 10000
;
```

Resultado:

```
2026-09-27T15:35:29.015996Z LOG pgloader version "3.6.10~devel"
2026-09-27T15:35:29.251945Z LOG Migrating from
mysql://root@127.0.0.1:3306/employees
2026-09-27T15:35:29.251945Z LOG Migrating into
pgsql://fernando@127.0.0.1:5432/pdb_employees
KABOOM!
SIMPLE-ERROR: pgloader failed to find schema "employees" in target
catalog.
```

Se agregó una cláusula `ALTER TABLE NAMES MATCHING ~/./ SET SCHEMA 'public'` para redirigir el esquema, pero el error persistió idéntico. **Causa identificada:** pgloader, al migrar desde MySQL/MariaDB, busca o crea en PostgreSQL un esquema con el mismo nombre que la base de origen (`employees`). Como en `pdb_employees` las tablas ya existen en el esquema `public` (creado así en la actividad anterior), y el modo `data only` no crea ni redirige esquemas —solo copia datos hacia objetos que ya deben existir exactamente donde pgloader los espera—, la migración no pudo continuar.

Decisión: en lugar de forzar el comportamiento de pgloader, se optó por el método de exportación/importación manual (`mysqldump`/cliente `mariadb` + `COPY`), que la propia consigna ofrece como alternativa y que da control total sobre el esquema destino.

1.1 Exportación de los datos desde MariaDB

Se exportó cada tabla a un archivo de texto delimitado por tabuladores, usando el cliente `mariadb` en modo batch (compatible con el formato TEXT de `COPY`, incluyendo `\N` para

valores NULL):

```
cd ~/tecBD1
mkdir -p datos_export
export MYSQL_PWD='********'

for t in departments employees dept_emp dept_manager salaries titles;
do
  mariadb -h 127.0.0.1 -u root --batch --skip-column-names employees \
    -e "SELECT * FROM $t;" > datos_export/$t.tsv
  echo "$t: $(wc -l < datos_export/$t.tsv) filas exportadas"
done

unset MYSQL_PWD
ls -lh datos_export/
```

Salida:

```
departments: 9 filas exportadas
employees: 300024 filas exportadas
dept_emp: 331603 filas exportadas
dept_manager: 24 filas exportadas
salaries: 2844047 filas exportadas
titles: 443308 filas exportadas

total 135M
-rw-r--r-- 1 fernando fernando 153 sep 27 11:39 departments.tsv
-rw-r--r-- 1 fernando fernando  11M sep 27 11:39 dept_emp.tsv
-rw-r--r-- 1 fernando fernando 816 sep 27 11:39 dept_manager.tsv
-rw-r--r-- 1 fernando fernando  14M sep 27 11:39 employees.tsv
-rw-r--r-- 1 fernando fernando  95M sep 27 11:40 salaries.tsv
-rw-r--r-- 1 fernando fernando  17M sep 27 11:40 titles.tsv
```

1.2 Importación de los datos en PostgreSQL

Se importó con `\copy` (comando de cliente de `psql`, no requiere permisos de archivo en el servidor), respetando el orden de dependencia de las llaves foráneas: primero las tablas padre (`departments`, `employees`), después las que dependen de ellas.

```
cd ~/tecBD1
export PGPASSWORD='********'

psql -h 127.0.0.1 -U fernando -d pdb_employees << 'EOF'
\copy departments FROM 'datos_export/departments.tsv' WITH (FORMAT
text)
\copy employees FROM 'datos_export/employees.tsv' WITH (FORMAT text)
\copy dept_emp FROM 'datos_export/dept_emp.tsv' WITH (FORMAT text)
\copy dept_manager FROM 'datos_export/dept_manager.tsv' WITH (FORMAT
text)
\copy salaries FROM 'datos_export/salaries.tsv' WITH (FORMAT text)
\copy titles FROM 'datos_export/titles.tsv' WITH (FORMAT text)
EOF

unset PGPASSWORD
```

Salida:

```
COPY 9
COPY 300024
COPY 331603
COPY 24
COPY 2844047
COPY 443308
```

Como las llaves foráneas de `pdb_employees` estaban **activas** durante la carga (no se usó `disable triggers`), cada fila fue validada contra sus restricciones de integridad referencial en el momento de insertarse. Que las 6 tablas se hayan cargado sin ningún error es, de por sí, una primera confirmación de que no existen referencias huérfanas en el dataset de origen.

1.3 Verificación del conteo de filas

Se consultó el conteo de cada tabla en ambos motores, de forma independiente, mediante una consulta SQL estándar con `UNION ALL` ejecutada tanto en MariaDB como en PostgreSQL.

Tabla 2

Comparación de conteo de filas entre MariaDB y PostgreSQL

Tabla	MariaDB (origen)	PostgreSQL (destino)	Coincide
departments	9	9	Sí
employees	300 024	300 024	Sí
dept_emp	331 603	331 603	Sí
dept_manager	24	24	Sí
salaries	2 844 047	2 844 047	Sí
titles	443 308	443 308	Sí
Total	3 919 015	3 919 015	Sí

Los 6 conteos son idénticos entre ambos motores. La migración de tablas se considera exitosa y completa.

Punto 2: Migración de Vistas

2.1 Extracción de las definiciones originales desde MariaDB

```
mariadb -h 127.0.0.1 -u root -t employees -e "
SELECT table_name, view_definition
FROM information_schema.views
WHERE table_schema='employees';"
```

Salida:

```
dept_emp_latest_date:
  select emp_no, max(from_date) as from_date, max(to_date) as to_date
  from employees.dept_emp group by emp_no

current_dept_emp:
  select l.emp_no, d.dept_no, l.from_date, l.to_date
  from (employees.dept_emp d join employees.dept_emp_latest_date l
  on (d.emp_no = l.emp_no and d.from_date = l.from_date
  and l.to_date = d.to_date))
```

2.2 Adaptación de la sintaxis a PostgreSQL

Tabla 3

Adaptación de la sintaxis de las vistas de MariaDB a PostgreSQL

Elemento en MariaDB	Adaptación en PostgreSQL	Motivo
Identificadores entre comillas invertidas	Sin comillas	PostgreSQL no las requiere para estos nombres
Prefijo employees.dept_emp en cada columna	dept_emp	El esquema public ya está en el search_path
MAX(), GROUP BY, JOIN ... ON	Igual	Sintaxis estándar SQL, sin diferencias

No fue necesario adaptar tipos de datos ni funciones: ambas vistas solo usan agregación (`MAX`), agrupación y una unión (`JOIN`), operaciones idénticas en ambos SGBD.

Definición adaptada (idéntica a la usada en el DDL de la actividad anterior):

```
CREATE VIEW dept_emp_latest_date AS
SELECT emp_no, MAX(from_date) AS from_date, MAX(to_date) AS to_date
FROM dept_emp
GROUP BY emp_no;

CREATE VIEW current_dept_emp AS
SELECT l.emp_no, d.dept_no, l.from_date, l.to_date
FROM dept_emp d
JOIN dept_emp_latest_date l
  ON d.emp_no = l.emp_no AND d.from_date = l.from_date AND l.to_date =
d.to_date;
```

2.3 Creación y prueba en PostgreSQL

Las vistas ya existían en `pdb_employees` (creadas como parte del DDL de la actividad anterior, cuando la base aún no tenía datos). Se confirmó su existencia y, con los datos ya cargados, se verificó que devuelven resultados correctos:

```
psql -h 127.0.0.1 -U fernando -d pdb_employees -c "\dv"
```

```
                  Listado de vistas
 Esquema |        Nombre        | Tipo  |  Dueño
---------+----------------------+-------+----------
 public  | current_dept_emp     | vista | fernando
 public  | dept_emp_latest_date | vista | fernando
(2 filas)
```

```
SELECT COUNT(*) FROM dept_emp_latest_date;   --
total_dept_emp_latest_date
SELECT COUNT(*) FROM current_dept_emp;       -- total_current_dept_emp
SELECT * FROM current_dept_emp LIMIT 5;
SELECT * FROM dept_emp_latest_date LIMIT 5;
```

Salida:

```
total_dept_emp_latest_date: 300024   |   total_current_dept_emp: 300024

current_dept_emp (muestra):
 emp_no | dept_no | from_date  |  to_date
  10004 | d004    | 1986-12-01 | 9999-01-01
  10018 | d004    | 1992-07-29 | 9999-01-01
  10020 | d004    | 1997-12-30 | 9999-01-01
  10025 | d005    | 1987-08-17 | 1997-10-15
  10027 | d005    | 1995-04-02 | 9999-01-01

dept_emp_latest_date (muestra):
 emp_no | from_date  |  to_date
  10001 | 1986-06-26 | 9999-01-01
  10002 | 1996-08-03 | 9999-01-01
  10003 | 1995-12-03 | 9999-01-01
  10004 | 1986-12-01 | 9999-01-01
  10005 | 1989-09-12 | 9999-01-01
```

Ambas vistas devuelven 300 024 filas —una por empleado—, consistente con el total de `employees`. La migración de vistas se considera exitosa.

Punto 3: Consultas de Verificación

3.1 Consultas SQL utilizadas

Se aplicaron tres capas de verificación, de menor a mayor exigencia.

a) Agregados de control (detectan truncamiento o corrupción de valores numéricos y de fecha):

```
SELECT
  (SELECT SUM(salary) FROM salaries) AS suma_salarios,
  (SELECT MIN(salary) FROM salaries) AS min_salario,
  (SELECT MAX(salary) FROM salaries) AS max_salario,
  (SELECT MIN(birth_date) FROM employees) AS min_nacimiento,
  (SELECT MAX(hire_date) FROM employees) AS max_contratacion,
  (SELECT COUNT(DISTINCT gender) FROM employees) AS generos_distintos;
```

b) Checksum MD5 (prueba de fidelidad byte a byte en las tablas pequeñas; en las tablas grandes —salaries, titles, dept_emp— un checksum por concatenación es poco práctico, por lo que ahí la verificación se apoya en el conteo exacto y los agregados):

```
-- MariaDB
SELECT MD5(GROUP_CONCAT(CONCAT_WS('|', dept_no, dept_name)
  ORDER BY dept_no SEPARATOR '\n')) FROM departments;
SELECT MD5(GROUP_CONCAT(CONCAT_WS('|', emp_no, dept_no, from_date,
to_date)
  ORDER BY emp_no, dept_no SEPARATOR '\n')) FROM dept_manager;

-- PostgreSQL
SELECT md5(string_agg(concat_ws('|', dept_no, dept_name), E'\n'
  ORDER BY dept_no)) FROM departments;
SELECT md5(string_agg(concat_ws('|', emp_no, dept_no, from_date::text,
to_date::text),
  E'\n' ORDER BY emp_no, dept_no)) FROM dept_manager;
```

c) Integridad referencial (búsqueda de filas huérfanas mediante LEFT JOIN, sintaxis idéntica en ambos motores):

```
SELECT 'dept_emp->employees' AS verificacion, COUNT(*) AS huerfanos
FROM dept_emp de LEFT JOIN employees e ON de.emp_no=e.emp_no WHERE
e.emp_no IS NULL
UNION ALL
SELECT 'dept_emp->departments', COUNT(*)
FROM dept_emp de LEFT JOIN departments d ON de.dept_no=d.dept_no WHERE
d.dept_no IS NULL
UNION ALL
SELECT 'dept_manager->employees', COUNT(*)
FROM dept_manager dm LEFT JOIN employees e ON dm.emp_no=e.emp_no WHERE
e.emp_no IS NULL
UNION ALL
SELECT 'dept_manager->departments', COUNT(*)
FROM dept_manager dm LEFT JOIN departments d ON dm.dept_no=d.dept_no
WHERE d.dept_no IS NULL
```

```
UNION ALL
SELECT 'salaries->employees', COUNT(*)
FROM salaries s LEFT JOIN employees e ON s.emp_no=e.emp_no WHERE
e.emp_no IS NULL
UNION ALL
SELECT 'titles->employees', COUNT(*)
FROM titles t LEFT JOIN employees e ON t.emp_no=e.emp_no WHERE e.emp_no
IS NULL;
```

3.2 Comparación de resultados entre motores

Tabla 4

Comparación de agregados de control entre motores

Métrica	MariaDB	PostgreSQL	Coincide
Suma de salarios	181 480 757 419	181 480 757 419	Sí
Salario mínimo	38 623	38 623	Sí
Salario máximo	158 220	158 220	Sí
Fecha de nacimiento mínima	1952-02-01	1952-02-01	Sí
Fecha de contratación máxima	2000-01-28	2000-01-28	Sí
Géneros distintos	2	2	Sí

Tabla 5

Comparación de checksums MD5 entre motores

Tabla	MariaDB	PostgreSQL	Coincide
departments	928233a3d60d594290245740b14abd86	928233a3d60d594290245740b14abd86	Idéntico
dept_manager	20778630383cc2cb498002cd6777c4fd	20778630383cc2cb498002cd6777c4fd	Idéntico

Tabla 6

Resultados de la verificación de integridad referencial (huérfanos)

Relación	MariaDB	PostgreSQL
dept_emp → employees	0	0
dept_emp → departments	0	0
dept_manager → employees	0	0
dept_manager → departments	0	0
salaries → employees	0	0
titles → employees	0	0

3.3 Análisis de resultados

Coincidencias: los tres niveles de verificación —conteos, checksums y aritmética de agregados— dieron resultados **idénticos** entre MariaDB y PostgreSQL. Los checksums MD5 de `departments` y `dept_manager` coinciden carácter por carácter, lo que confirma que

esas tablas se replicaron sin ninguna alteración. Las seis verificaciones de integridad referencial no encontraron ninguna fila huérfana en ningún motor, confirmando que el dataset de origen es internamente consistente y que la migración no introdujo ni rompió ninguna relación.

Problema encontrado y solución: al ejecutar la consulta de huérfanos contra las tablas grandes (`salaries`, con 2.8 millones de filas), PostgreSQL devolvió el siguiente error:

```
ERROR:  could not resize shared memory segment "/PostgreSQL.2687711846"
to 8388608 bytes: No space left on device
```

Diagnóstico: no fue un problema de espacio en disco de la VM (`/dev/shm` mostraba 2.3 GB libres a nivel de sistema operativo). La causa es que **Docker limita por defecto la memoria compartida (`/dev/shm`) de cada contenedor a 64 MB**, independientemente de la RAM disponible en el host. Al intentar paralelizar el `JOIN` sobre una tabla de millones de filas, PostgreSQL solicitó un segmento de memoria compartida mayor a ese límite del contenedor.

Solución aplicada: se desactivó la paralelización solo para la sesión de verificación, sin modificar la configuración del contenedor:

```
SET max_parallel_workers_per_gather = 0;
```

Con esto, la consulta se ejecutó en un único proceso y completó sin error, devolviendo 0 huérfanos en las 6 relaciones. Una alternativa más permanente, no aplicada aquí por no ser necesaria para esta actividad, sería aumentar el tamaño de `/dev/shm` del contenedor `postgresql` en `docker-compose.yml` mediante la opción `shm_size`.

Conclusión general del Punto 3: la migración de datos de `employees` (MariaDB) a `pdb_employees` (PostgreSQL) es completa, correcta y verificable en múltiples niveles independientes. No se detectaron pérdidas, truncamientos, duplicados ni relaciones rotas.

Backup de PostgreSQL

Se generó un respaldo de la base `pdb_employees` ya migrada, en formato custom de `pg_dump` (comprimido):

```
cd ~/tecBD1
pg_dump -h 127.0.0.1 -U fernando -d pdb_employees -F c -f
pdb_employees_backup.dump
ls -lh pdb_employees_backup.dump
```

```
-rw-r--r-- 1 fernando fernando 34M sep 27 11:51
pdb_employees_backup.dump
```

Verificación del contenido del backup (sin restaurarlo):

```
pg_restore -l pdb_employees_backup.dump | head -20
```

```
;
; Archive created at 2026-09-27 11:50:51 -04
;     dbname: pdb_employees
;     TOC Entries: 66
;     Compression: gzip
;     Format: CUSTOM
;     Dumped from database version: 18.6 (Debian 18.6-1.pgdg13+2)
;     Dumped by pg_dump version: 18.6 (Debian 18.6-1.pgdg12+2)
;
; Selected TOC Entries:
;
859; 1247 16486 TYPE public gender_enum fernando
221; 1259 16515 TABLE public dept_emp fernando
3502; 0 0 COMMENT public TABLE dept_emp fernando
```

El backup contiene 66 entradas en el TOC, incluyendo el tipo `gender_enum`, las tablas con sus datos y los comentarios documentados en la actividad anterior, confirmando que el respaldo preserva tanto los datos como la estructura y su documentación.

> *Nota: también se generó un respaldo en texto plano (pdb_employees_backup.sql, 135 MB) como copia local adicional. Solo el .dump (34 MB) se incluye en el repositorio de GitHub, ya que GitHub rechaza archivos mayores a 100 MB en un push estándar.*

Conclusiones

La fase de migración de datos, vistas y verificación del proyecto `employees` → `pdb_employees` se completó exitosamente. Los tres puntos de la consigna quedaron cubiertos con evidencia verificable: 3 919 015 filas migradas sin discrepancias en 6 tablas, 2 vistas recreadas y probadas con datos reales, y una verificación en tres capas (conteos,

checksums MD5 e integridad referencial) que no encontró ninguna diferencia entre el origen y el destino. El proceso incluyó dos desvíos técnicos documentados —el comportamiento de pgloader respecto al nombre del esquema destino, y el límite de memoria compartida de Docker— ambos diagnosticados y resueltos sin comprometer la integridad de los datos.

Referencias

Docker Inc. (s.f.). docker container run. Docker Docs. https://docs.docker.com/reference/cli/docker/container/run/

Lentz, G., & Bales, P. (s.f.). test_db: Sample MySQL database [Base de datos de ejemplo]. GitHub. https://github.com/datacharmer/test_db

pgloader. (s.f.). MySQL database migration. pgloader Documentation. https://pgloader.readthedocs.io/en/latest/ref/mysql.html

PostgreSQL Global Development Group. (s.f.). COPY. PostgreSQL 18 Documentation. https://www.postgresql.org/docs/18/sql-copy.html

PostgreSQL Global Development Group. (s.f.). pg_dump. PostgreSQL 18 Documentation. https://www.postgresql.org/docs/18/app-pgdump.html

Anexos

Archivos entregados junto con este informe, en el repositorio de GitHub:

1. informe_final.md — informe en formato Markdown (fuente de este documento).
2. backup/pdb_employees_backup.dump — respaldo de PostgreSQL en formato custom.
3. README.md — descripción del contenido del repositorio.